Nome e Cognome: _____ **Matricola:** _____

Istruzioni: Il tempo a disposizione per completare la prova è di **45 minuti**. Durante la prova, non è consentito consultare materiale didattico, né interagire con dispositivi elettronici o colleghi. Nel caso di domande a risposta multipla con più risposte corrette, ciascuna risposta corretta indicata vale una corrispondente frazione del punteggio complessivo, mentre le risposte errate comportano una pari penalizzazione. In nessun caso, il punteggio complessivo relativo a una domanda sarà negativo.

Es. **1.** **(2 punti)** Quali delle seguenti affermazioni su Internet e il World Wide Web sono **vere**?

- **A** Internet è la rete di reti che collega i dispositivi; il Web è un insieme di documenti ipertestuali collegati fra loro.
- **B** Il Web è un servizio che funziona *sopra* Internet, insieme ad altri (posta elettronica, trasferimento di file, ...).
- C La posta elettronica è una parte del World Wide Web.
- **D** Le risorse del Web sono identificate da URL e trasferite con il protocollo HTTP.
- E Un documento è nel Web solo se è scritto in HTML5.
- F Il Web usa un protocollo di rete alternativo a TCP/IP.

Risposte: A, B, D. Internet è l'infrastruttura (la rete di reti basata su TCP/IP); il World Wide Web è uno dei servizi che la usano: documenti ipertestuali identificati da URL e trasferiti con HTTP. Anche la posta elettronica usa Internet, ma non fa parte del Web. *(Dispensa: cap. 1, «Internet e il World Wide Web»)*

Es. **2.** **(2 punti)** Quali coppie di elementi sono corrette per una **lista di descrizione** (glossario)?

- A `<dl>` per la lista, `<li>` per il termine e `<dd>` per la descrizione.
- B `<list>` per la lista, `<term>` per il termine e `<desc>` per la descrizione.
- C `<ol>` per la lista, `<dt>` per il termine e `<dd>` per la descrizione.
- D `<dd>` per la lista, `<dl>` per il termine e `<dt>` per la descrizione.
- E `<ul>` per la lista, `<li>` per il termine e `<p>` per la descrizione.
- **F** `<dl>` per la lista, `<dt>` per il termine e `<dd>` per la descrizione.

Risposta: F. Le liste di descrizione associano termini e definizioni: `dl` (description list) contiene coppie `dt` (termine) e `dd` (descrizione). Le liste puntate sono `ul`, quelle numerate `ol`, entrambe con elementi `li`. *(Dispensa: cap. 2, «Liste di descrizione»)*

Es. **3.** **(2 punti)** Quali delle seguenti affermazioni sul contenuto generato con `::before` e `::after` sono **vere**?

- **A** Non fa parte del DOM: non si trova, per esempio, con `querySelector`.
- B Funziona solo sugli elementi vuoti come `<img>` e `<input>`.
- C Viene aggiunto al codice HTML della pagina inviata dal server.
- **D** Non dovrebbe contenere informazioni essenziali, perché non è vero contenuto.
- E Si può selezionare con `document.querySelector('::before')`.
- F Sostituisce il contenuto dell'elemento a cui si applica.

Risposte: A, D. `::before` e `::after` creano un figlio fittizio all'inizio o alla fine del contenuto dell'elemento, solo se `content` è impostato. Non esiste nel DOM e non è affidabile per lettori di schermo e copia-incolla: va usato per decorazioni, non per informazioni. *(Dispensa: cap. 3, «Pseudo-elementi»)*

Es. **4.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 const valori = [0, "0", "", [], null, NaN];
2 for (const v of valori) {
3   if (v) console.log(JSON.stringify(v));
4 }
```

- A `[]`.
- **B** `"0"`, `[]`.
- C `"0"`, `""`, `[]`.
- D Non viene stampato nulla.
- E `"0"`.
- F `0`, `"0"`, `[]`.

Risposta: B. Sono *falsy* `false`, `0`, `""`, `null`, `undefined` e `NaN`: tutto il resto è *truthy*, compresa la stringa `"0"` (non vuota) e l'array vuoto (un oggetto). *(Dispensa: cap. 5, «Operatori»)*

SOLUZIONI

Es. **5.** **(2 punti)** Quali delle seguenti affermazioni sui meccanismi di memorizzazione nel browser sono **vere**?

- **A** Esiste un `localStorage` distinto per ogni combinazione di dominio, protocollo e porta.
- B IndexedDB può memorizzare soltanto coppie di stringhe.
- C I dati in `localStorage` vengono cancellati alla chiusura del browser.
- **D** I cookie vengono inviati al server in tutte le richieste verso lo stesso dominio, tramite l'header `Cookie`.
- E `sessionStorage` è condiviso fra tutte le schede aperte sullo stesso sito.
- **F** `localStorage` memorizza solo valori stringa.

Risposte: A, D, F. I cookie fanno parte di HTTP e viaggiano con le richieste; `localStorage` e `sessionStorage` no, e possono quindi contenere molti più dati (solo stringhe). `localStorage` sopravvive anche al riavvio del browser, `sessionStorage` è legato alla singola scheda. IndexedDB è un vero database. Tutti questi dati sono sul computer dell'utente, che può leggerli e modificarli: il server non deve fidarsene. *(Dispensa: cap. 8, «Memorizzare dati nel browser»)*

Es. **6.** **(2 punti)** In PHP, come si legge il valore del campo `todo` di un form inviato con il metodo `POST`?

- A `$_GET['todo']`
- B `$request.body.todo`
- C `$_POST.todo()`
- **D** `$_POST['todo']`
- E `getenv('todo')`
- F `$_COOKIE['todo']`

Risposta: D. Uno dei vantaggi dello scripting lato server è la pre-elaborazione della richiesta: in PHP i parametri sono già negli array associativi `$_GET` e `$_POST`, e i cookie in `$_COOKIE`. *(Dispensa: cap. 10, «PHP»)*

Es. **7.** **(2 punti)** Qual è l'output dell'esecuzione del seguente modulo con Node.js? Il file `dati.txt` esiste e contiene la stringa `ciao`.

```
1 import fs from "fs";
2
3 console.log("inizio");
4 fs.readFile("dati.txt", "utf8", (err, testo) => console.log("asincrono:", testo));
5 const t = fs.readFileSync("dati.txt", "utf8");
6 console.log("sincrono:", t);
7 console.log("fine");
```

- A `inizio, sincrono: ciao, asincrono: ciao, fine.`
- B `inizio, fine, asincrono: ciao, sincrono: ciao.`
- C `inizio, fine, sincrono: ciao, asincrono: ciao.`
- **D** `inizio, sincrono: ciao, fine, asincrono: ciao.`
- E `inizio, sincrono: undefined, fine, asincrono: ciao.`
- F `inizio, asincrono: ciao, sincrono: ciao, fine.`

Risposta: D. `fs.readFile` è asincrona: avvia la lettura, che procede fuori dal thread di JavaScript, e ritorna subito; la callback viene eseguita dall'event loop dopo la fine del codice sincrono. `fs.readFileSync` invece blocca l'esecuzione finché il file non è stato letto, e restituisce il contenuto. In Node.js le operazioni sincrone sono l'eccezione. *(Dispensa: cap. 11, «Il modello di Node.js»)*

SOLUZIONI

Es. **8.** **(2 punti)** Si consideri la seguente applicazione Express, in ascolto sulla porta 3000. Che cosa rispondono due richieste `POST /form`, la prima con il corpo `msg=Ciao+mondo&num=42` e l'header `Content-Type: application/x-www-form-urlencoded`, la seconda con il corpo `{"msg": "Ciao", "num": 7}` e l'header `Content-Type: application/json`?

```
1  import express from "express";
2
3  const app = express();
4
5  app.use(express.urlencoded({ extended: false }));
6  app.post("/form", (req, res) => {
7    res.send(`msg=${req.body?.msg}; num=${req.body?.num}`);
8  });
9
10 app.listen(3000);
```

- **A** 200 `msg=Ciao mondo; num=42`, 200 `msg=undefined; num=undefined`.
- B 200 `msg=undefined; num=undefined`, 200 `msg=undefined; num=undefined`.
- C 200 `msg=undefined; num=undefined`, 200 `msg=Ciao; num=7`.
- D 200 `msg=Ciao+mondo; num=42`, 200 `msg=Ciao; num=7`.
- E 200 `msg=Ciao mondo; num=42`, 200 `msg=Ciao; num=7`.
- F 200 `msg=Ciao+mondo; num=42`, 200 `msg=undefined; num=undefined`.

Risposta: A. `express.urlencoded` analizza i corpi codificati come URL (quelli dei form, decodifica compresa), `express.json` quelli in JSON: è il `Content-Type` a decidere, e ciascuno ignora le richieste che non sono nel suo formato. Se nessun middleware analizza il corpo, `req.body` resta indefinito. *(Dispensa: cap. 15, «express.urlencoded» e «express.json»)*

Es. **9.** **(5 punti)** Si descriva il **box model** del CSS e si spieghi l'effetto della proprietà `box-sizing`, con un esempio numerico.

Traccia della risposta.

- Ogni elemento è un riquadro: contenuto, padding, bordo, margine (dall'interno).
- Il padding prende lo sfondo dell'elemento, il margine è spazio trasparente esterno.
- Con `content-box` (predefinito) `width` è la larghezza del contenuto: `width: 200px; padding: 20px; border: 5px` occupa 250px fino al bordo.
- Con `border-box` `width` arriva fino al bordo: gli stessi valori danno 200px, con un contenuto di 150px. Spesso si imposta `* { box-sizing: border-box; }`.

(Dispensa: cap. 4, «Il box model»)

Es. **10.** **(5 punti)** Si confrontino `innerHTML`, `outerHTML` e `textContent`, e si spieghi perché assegnare a `innerHTML` dati forniti dall'utente è pericoloso.

Traccia della risposta.

- `innerHTML` è il codice HTML contenuto nell'elemento, `outerHTML` quello dell'intero elemento (tag compresi), `textContent` solo il testo dei discendenti.
- Sono scrivibili: assegnare `innerHTML` sostituisce il contenuto, assegnare `outerHTML` l'elemento intero; la stringa passa dal parser HTML, che la interpreta (e la corregge) come farebbe con un file.
- Se la stringa contiene dati dell'utente, l'eventuale markup viene interpretato (`<script>`, attributi come `onerror`): è la via per il *Cross-Site Scripting* (XSS).
- Per inserire testo si usa `textContent` (o `innerText`), oppure si creano i nodi con `createElement` e `createTextNode`.

(Dispensa: cap. 7, «innerHTML e outerHTML»)

Es. **11.** **(5 punti)** Realizzando un'applicazione web con il solo modulo `http` di Node.js bisogna affrontare tre problemi: routing, templating e analisi del corpo delle richieste. Si descriva ciascuno e come lo si può risolvere.

Traccia della risposta.

- Routing: scegliere il codice che serve la richiesta in base a percorso (`pathname`, senza la query) e metodo; `404` se il percorso non esiste, `405` se il metodo non è previsto. Con uno `switch` diventa difficile con parametri nel percorso e controlli di accesso.
- Templating: generare l'HTML da un modello con segnaposto e dati (Pug: inclusioni, ereditarietà, locals con escaping, condizionali, cicli), invece di concatenare stringhe.
- Corpo: arriva a pezzi (eventi `data` ed `end`), va ricomposto e decodificato (`&`, `=`, `+` come spazio, `decodeURIComponent`).
- Dopo una `POST` conviene rispondere con un reindirizzamento (Post/Redirect/Get); le `GET` non devono modificare lo stato.

(Dispensa: cap. 12, «Routing», «Motori di template» e «Analizzare il corpo delle richieste»)

Griglia delle risposte

Es.	Risposta	Capitolo	Argomento
1	A, B, D	1	Internet e il Web
2	F	2	liste
3	A, D	3	pseudo-elementi
4	B	5	uguaglianza debole e stretta
5	A, D, F	8	memorizzare dati nel browser
6	D	10	scripting lato server
7	D	11	il modello di Node.js
8	A	15	middleware utili
9	aperta	4	box model
10	aperta	7	proprietà dei nodi
11	aperta	12	routing

Punteggio massimo: 16 (chiuse) + 15 (aperte) = **31 punti**. Nelle domande con più risposte corrette ogni risposta giusta indicata vale una frazione dei punti e ogni risposta sbagliata indicata ne toglie altrettanti; il punteggio di una domanda non scende mai sotto zero.

SOLUZIONI